

ESP32 Self-Balancing Robot

Closed-Loop Control Architecture & Production Firmware Report

Physical Specs: 1.3 kg mass, 22 cm height	Control Frequency: 100 Hz (PID_DT = 0.01s)
Sensors & Fusion: Adafruit MPU6050 + Complementary Filter	Actuation: Dual NEMA 17 + DRV8825 (1/16 Microstepping)

1. Executive Summary

An inverted pendulum is an inherently unstable dynamical system that requires continuous active state measurement and rapid counter-torque response to maintain an upright posture. This report details the complete embedded implementation for a 1.3 kg, 22 cm tall self-balancing robot. Powered by an **ESP32 microcontroller**, an **Adafruit MPU6050 IMU**, and dual **NEMA 17 stepper motors** driven via **DRV8825 drivers** using the **AccelStepper** library, the system provides smooth balancing, active calibration, dynamic serial PID tuning, and safety cutoffs.

2. Detailed Hardware Architecture & Components

The hardware components were selected to achieve fast torque generation, low-latency I2C polling at 400 kHz, and microstepping motor control without missed pulses:

ESP32	ESP32 Development Board (Master Controller) Dual-core Xtensa LX6 MCU running at 240 MHz. Handles 100 Hz PID execution, 400 kHz Fast I2C bus transactions, AccelStepper step pulse generation, and real-time UART telemetry/tuning.
MPU6050	MPU6050 6-DOF IMU (Accelerometer + Gyroscope) Configured for $\pm 2g$ accelerometer range, $\pm 250^\circ/s$ gyro range, and an onboard 21 Hz low-pass filter (DLPF) to filter mechanical resonance before sampling.
DRV8825	Dual DRV8825 Stepper Motor Drivers Configured for 1/16 microstepping (3200 steps/rev). Controlled through dedicated STEP, DIR, and an active-LOW Enable line (GPIO 27) for automatic power cutoffs.
NEMA 17	NEMA 17 Bipolar Stepper Motors Provide instant, backlash-free torque delivery up to 6,000 steps/sec (~112 RPM) with high acceleration profiles (12,000 steps/s ²) to arrest gravitational tipping.
Li-ion Battery	Power Delivery & Capacitive Filtering Powered by a high-discharge Li-ion battery pack with dedicated 100 μ F decoupling capacitors directly across the driver VMOT and GND pins to suppress switching back-EMF spikes.

3. Pin Mapping & Hardware Interface Table

Device / Module	Pin Function	ESP32 GPIO / Connection	Role in Code
MPU6050	SDA / SCL	GPIO 21 / GPIO 22	Fast I2C Bus (400 kHz)
Left Driver	STEP1 / DIR1	GPIO 18 / GPIO 19	stepper1 (AccelStepper::DRIVER)
Right Driver	STEP2 / DIR2	GPIO 25 / GPIO 26	stepper2 (AccelStepper::DRIVER)
Both Drivers	EN (Enable)	GPIO 27	Active-LOW motor power control
Both Drivers	MODE0, MODE1, MODE2	3.3V, 3.3V, GND	Hardware configured 1/16 microstepping
Both Drivers	RESET / SLEEP	Short + 3.3V Rail	Keep drivers awake and operating
Power Rail	VMOT / GND	12V Battery (+ / -)	Motor power rail with 100 μ F capacitors

4. Control Logic & Software Calibration

The software architecture incorporates active bias compensation, complementary filtering, and closed-loop PID control:

- Automated IMU Calibration:** During startup, `calibrateMPU()` averages 1,000 samples to establish the gyroscope Y-axis baseline bias and locks in the natural balanced center-of-mass angle (`targetAngle`).
- Complementary Filter:** Fuses the integrated gyro angular rate with the gravity-referenced accelerometer angle:

$$\theta_{current} = 0.98 \cdot (\theta_{current} + \omega_{gyro} \cdot \Delta t) + 0.02 \cdot \theta_{accel}$$

- **Noise Deadband & Integral Anti-Windup:** Ignores angular errors smaller than 0.15° to prevent continuous idle jitter, while constraining the integral accumulator (errorSum) between -50.0 and +50.0.
- **Fall Detection Safety Cutoff:** If tilt error exceeds 35°, the ESP32 sets the enable pin (GPIO 27) HIGH, cutting coil power to protect the motors and drivers.
- **Live Serial Telemetry & Tuning:** Supports real-time UART coefficient updates without recompiling firmware (e.g., kp900, ki0.6, kd2.0, t-1.5).

5. Complete Firmware Source Code (ESP32 / AccelStepper)

The full, production C++ code utilized on the physical robot:

```
/*
  SELF-BALANCING ROBOT — ESP32 + DRV8825 + MPU6050 + AccelStepper
  light (~1.3kg) bot, 22cm tall.

  WIRING:
  STEP1 -> GPIO18   DIR1 -> GPIO19
  STEP2 -> GPIO25   DIR2 -> GPIO26
  EN     -> GPIO27
  MPU6050: SDA -> GPIO21, SCL -> GPIO22
  DRV8825 microstepping to 1/16
*/

#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <AccelStepper.h>

// PIN DEFINITIONS
#define STEP_PIN_1 18
#define DIR_PIN_1  19
#define STEP_PIN_2 25
#define DIR_PIN_2  26
#define EN_PIN     27

AccelStepper stepper1(AccelStepper::DRIVER, STEP_PIN_1, DIR_PIN_1);
AccelStepper stepper2(AccelStepper::DRIVER, STEP_PIN_2, DIR_PIN_2);
Adafruit_MPU6050 mpu;

float Kp = 900.0;
float Ki = 0.6;
float Kd = 2.0;

float targetAngle = 0.0;    // calibrated upright angle
const float DEADBAND = 0.15; // degrees; ignore tiny noise inside this band

// 1/16 microstepping -> 3200 steps/rev.
const float MAX_STEPS_PER_SEC = 6000.0; // ~112 RPM cap
const float MAX_ACCEL_STEPS   = 12000.0;

const float FALL_ANGLE = 35.0; // beyond this, cut motors

// STATE
float currentAngle = 0.0;
float gyroBiasY = 0.0;
float error = 0.0, lastError = 0.0, errorSum = 0.0;
float pidOutput = 0.0;
bool motorsEnabled = false;

unsigned long lastTime = 0;
unsigned long pidTimer = 0;
unsigned long debugTimer = 0;
const float PID_DT = 0.01; // 100Hz loop

void setup() {
  Serial.begin(115200);
  delay(300);

  pinMode(EN_PIN, OUTPUT);
  digitalWrite(EN_PIN, HIGH); // keep motors OFF until calibration finishes

  stepper1.setMaxSpeed(MAX_STEPS_PER_SEC);
  stepper2.setMaxSpeed(MAX_STEPS_PER_SEC);
  stepper1.setAcceleration(MAX_ACCEL_STEPS);
  stepper2.setAcceleration(MAX_ACCEL_STEPS);

  Wire.begin(21, 22);
}
```

```

Wire.setClock(400000);

if (!mpu.begin()) {
    Serial.println("MPU6050 not found. Check wiring.");
    while (1) delay(10);
}
mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
mpu.setGyroRange(MPU6050_RANGE_250_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

calibrateMPU();

Serial.println("Calibration done. Hold the bot upright to enable motors.");
Serial.println("Serial tuning: type e.g. kp70 ki0.8 kd3.2 t-1.5 (t = target offset)");

lastTime = micros();
pidTimer = millis();
debugTimer = millis();
}

void calibrateMPU() {
    Serial.println("Calibrating... keep the bot still and BALANCED upright.");
    delay(1500);

    double gyroSum = 0, angleSum = 0;
    const int N = 1000;
    for (int i = 0; i < N; i++) {
        sensors_event_t a, g, temp;
        mpu.getEvent(&a, &g, &temp);
        gyroSum += g.gyro.y;
        angleSum += atan2(-a.acceleration.x,
                        sqrt(a.acceleration.y * a.acceleration.y + a.acceleration.z * a.acceleration.z))
                        * 180.0 / M_PI;
        delay(2);
    }
    gyroBiasY = gyroSum / N;
    targetAngle = angleSum / N;
    currentAngle = targetAngle;

    Serial.print("Gyro Y bias: "); Serial.println(gyroBiasY, 4);
    Serial.print("Target (balanced) angle: "); Serial.println(targetAngle, 2);
}

void loop() {
    readSerialTuning();

    if (millis() - pidTimer >= 10) {
        pidTimer = millis();

        sensors_event_t a, g, temp;
        mpu.getEvent(&a, &g, &temp);

        unsigned long now = micros();
        float dt = (now - lastTime) / 1000000.0;
        lastTime = now;

        float accelAngle = atan2(-a.acceleration.x,
                                sqrt(a.acceleration.y * a.acceleration.y + a.acceleration.z * a.acceleration.z))
                                * 180.0 / M_PI;
        float gyroRate = g.gyro.y - gyroBiasY;
        currentAngle = 0.98 * (currentAngle + gyroRate * 180.0 / M_PI * dt) + 0.02 * accelAngle;

        error = currentAngle - targetAngle;

        // Fall detection: cut motor power, don't just zero speed
        if (abs(error) > FALL_ANGLE) {
            disableMotors();
            errorSum = 0;
            lastError = 0;
            return;
        } else if (!motorsEnabled) {
            enableMotors();
        }

        // Deadband: ignore sensor noise near target
        float effectiveError = (abs(error) < DEADBAND) ? 0.0 : error;

        errorSum += effectiveError * PID_DT;
        errorSum = constrain(errorSum, -50.0, 50.0); // anti-windup

        float dError = (effectiveError - lastError) / PID_DT;
        lastError = effectiveError;

        pidOutput = (Kp * effectiveError) + (Ki * errorSum) + (Kd * dError);
        pidOutput = constrain(pidOutput, -MAX_STEPS_PER_SEC, MAX_STEPS_PER_SEC);
    }
}

```

```

    stepper1.setSpeed(pidOutput);
    stepper2.setSpeed(-pidOutput); // reversed mounting
}

stepper1.runSpeed();
stepper2.runSpeed();

if (millis() - debugTimer >= 200) {
    debugTimer = millis();
    Serial.print("Angle: "); Serial.print(currentAngle, 2);
    Serial.print(" | Err: "); Serial.print(error, 2);
    Serial.print(" | Out: "); Serial.print(pidOutput, 0);
    Serial.print(" | Kp="); Serial.print(Kp, 1);
    Serial.print(" Ki="); Serial.print(Ki, 2);
    Serial.print(" Kd="); Serial.println(Kd, 2);
}
}

void enableMotors() {
    digitalWrite(EN_PIN, LOW); // DRV8825 EN is active LOW
    motorsEnabled = true;
}

void disableMotors() {
    digitalWrite(EN_PIN, HIGH);
    stepper1.setSpeed(0);
    stepper2.setSpeed(0);
    motorsEnabled = false;
}

void readSerialTuning() {
    if (!Serial.available()) return;
    String cmd = Serial.readStringUntil('
');
    cmd.trim();
    if (cmd.length() < 3) return;

    String prefix = cmd.substring(0, 2);
    float val = cmd.substring(2).toFloat();

    if (prefix == "kp") { Kp = val; Serial.print("Kp set to "); Serial.println(Kp); }
    else if (prefix == "ki") { Ki = val; Serial.print("Ki set to "); Serial.println(Ki); }
    else if (prefix == "kd") { Kd = val; Serial.print("Kd set to "); Serial.println(Kd); }
    else if (cmd.charAt(0) == 't') {
        targetAngle += cmd.substring(1).toFloat();
        Serial.print("Target angle set to "); Serial.println(targetAngle);
    }
}
}

```